UDP Server Experiment

1. UDP Overview

UDP (User Datagram Protocol) is a **connectionless, unreliable but efficient** transport layer protocol. Unlike TCP, UDP doesn't establish connections, doesn't guarantee delivery, doesn't guarantee order, but **low overhead, low latency**, very suitable for scenarios with high real-time requirements and tolerable occasional packet loss.

In IoT applications, UDP is widely used for **high-frequency sensor data reporting** (e.g., temperature/humidity sampled 100 times per second, losing a few frames is fine), **LAN device discovery** (broadcast/multicast to detect online devices in same subnet), **DNS domain name resolution** (small query volume, timeout retry works), and **real-time audio/video streaming** (latency priority over quality integrity). Mastering UDP's `sendto()`/`recvfrom()` programming model is the foundation for advanced experiments like custom protocol packing, real-time data reporting and LAN device discovery.

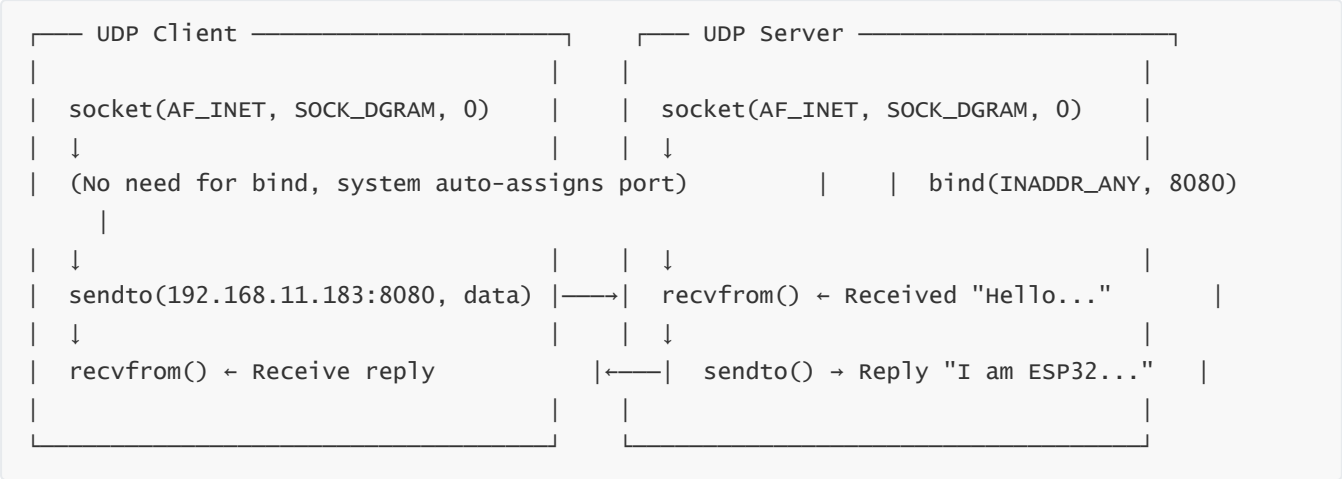
Feature	UDP	TCP
Connection	Connectionless, send directly	Three-way handshake to establish connection
Reliability	No guarantee of delivery, no guarantee of order	Guarantee delivery, guarantee order
Transmission Efficiency	High (8-byte header)	Lower (20-byte header)
Typical Use	Video streaming, DNS, IoT sensor data	Web, file transfer, MQTT

Feature	UDP	TCP
Core APIs	`sendto()` / `recvfrom()`	`send()` / `recv()`

> **This project contains UDP client and server two sub-examples**: client sends data periodically and receives server replies; server listens on port, automatically replies to received data.

2. Core Concepts

2.1 UDP Socket Communication Flow



2.2 `sendto()` and `recvfrom()`

`sendto()` and `recvfrom()` are the two core APIs for UDP programming - the former sends datagrams with target address, the latter receives data while getting sender address for reply.

```
// Send: specify target IP and port
ssize_t sendto(int sockfd, const void *buf, size_t len, int flags,
               const struct sockaddr *dest_addr, socklen_t addrlen);

// Receive: also get sender address (for reply)
ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags,
                 struct sockaddr *src_addr, socklen_t *addrlen);
```

2.3 Non-Blocking Receive (MSG_DONTWAIT)

`MSG_DONTWAIT` flag makes `recvfrom()` non-blocking - returns -1 immediately (errno=EAGAIN) when no data without blocking task. This example uses 10ms high-frequency polling, ensuring low-latency receive while not affecting timed sending:

```
int recv_len = recvfrom(sock_fd, recv_buf, sizeof(recv_buf)-1,
                        // Non-blocking flag
                        MSG_DONTWAIT,
                        (struct sockaddr *)&from_addr, &len);
```

2.4 Network Byte Order

Network protocols use **Big-Endian** uniformly, while ESP32 is little-endian CPU. Ports and IP addresses must be converted - `htons()` converts port number, `inet_pton()` converts IP string to network-order binary:

```
// Port to network byte order
dest_addr.sin_port = htons(8080);
inet_pton(AF_INET, "192.168.1.1", &addr);
```

---

3. Project Architecture and Flow

3.1 UDP Server

```
app_main()
  ├─ wifi_init()                        ← WiFi STA initialization (wait for IP)
  │
  └─ xTaskCreate(vUdpServerTask, ...)   ← Create UDP server task
      └─ vUdpServerTask():
          ├─ socket(AF_INET, SOCK_DGRAM, 0)  → Create UDP socket
          ├─ bind(INADDR_ANY, 8080)          → Bind listening port
          └─ while(1):
              ├─ recvfrom(MSG_DONTWAIT) ← Non-blocking receive client data
              ├─ If valid data received → sendto() auto reply
              └─ vTaskDelay(10ms)
```

---

4. Hardware Dependencies

- **Hardware Connection**: No external wiring, WiFi wireless communication
- **Client Dependent Components**: `lwip/sockets`, `esp_wifi`, `esp_netif`, `esp_event`, `nvs_flash`, `FreeRTOS`
- **Server Dependent Components**: Same as above, additional `lwip/netdb`

---

5. Source Code Analysis — UDP Server

Complete source code: `Source Code → ESP-IDF → 5.Network_Protocol → 3.UDP → UDP_Server`

5.1 vUdpServerTask() — Listen and Auto Reply

The biggest difference between UDP server and client is just one line of code: **server must `bind()` port**. Without bind, OS doesn't know which Socket to throw data to. When binding, use `INADDR_ANY` - meaning "regardless of which network interface the message came from, as long as it's this port, receive it".

`recvfrom()` has a benefit: it not only receives the data, but also **automatically fills in the sender's address** (`client_addr`). Server gets this address and can directly `sendto()` back, no need to pre-configure target address like client does.

```c
void vUdpServerTask(void *pv)
{
    int sock_fd;
    struct sockaddr_in server_addr = {0};

    // Create UDP socket
    sock_fd = socket(AF_INET, SOCK_DGRAM, 0);

    // Bind port
    server_addr.sin_family = AF_INET;
    // Listen all IP
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(UDP_SERVER_PORT);  // Port number
    bind(sock_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));

    while (1)
    {
struct sockaddr_in client_addr;             // Client address
        socklen_t client_len = sizeof(client_addr);
        char recv_buf[128] = {0};

        // Non-blocking receive
        int recv_len = recvfrom(sock_fd, recv_buf, sizeof(recv_buf)-1,
                                MSG_DONTWAIT,
                                (struct sockaddr *)&client_addr, &client_len);
        if (recv_len > 0)
        {
            recv_buf[recv_len] = '\0';
            ESP_LOGI(TAG, "Receive computer messages: %s", recv_buf);

            // Auto reply
            char reply_msg[] = "I am ESP32 UDP Server!";
            // Reply to client
            sendto(sock_fd, reply_msg, strlen(reply_msg), 0,
                   (struct sockaddr *)&client_addr, client_len);
        }

        vTaskDelay(10);
    }
    close(sock_fd);
    vTaskDelete(NULL);
}
```

6. Operation Flow

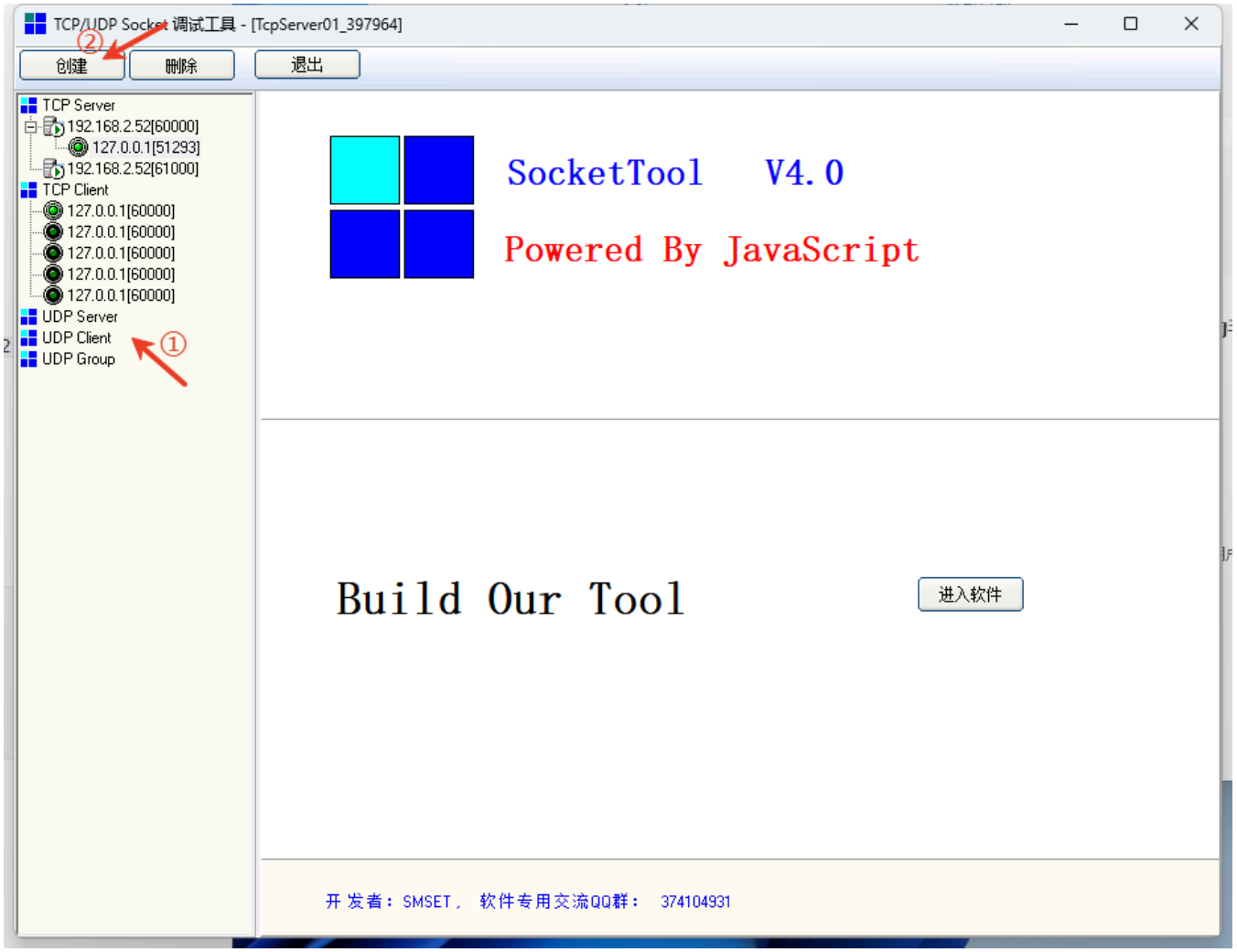
6.1 Compilation and Flashing Flow

ESP-IDF flashing flow: See `ESP-IDF → 1. Environment Setup → 3. Flashing Flow → Flashing Flow`.

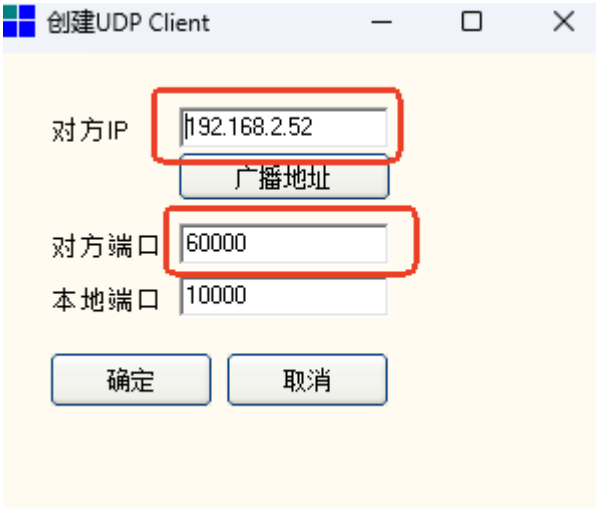
6.2 Test Method

Flash UDP_Server to ESP32, PC side sends data to ESP32's IP port 8080, ESP32 automatically replies `I am ESP32 UDP Server!`. For convenience, we can use another network debug assistant for the server side.

Click page to open, ① select UDP Client ② select create

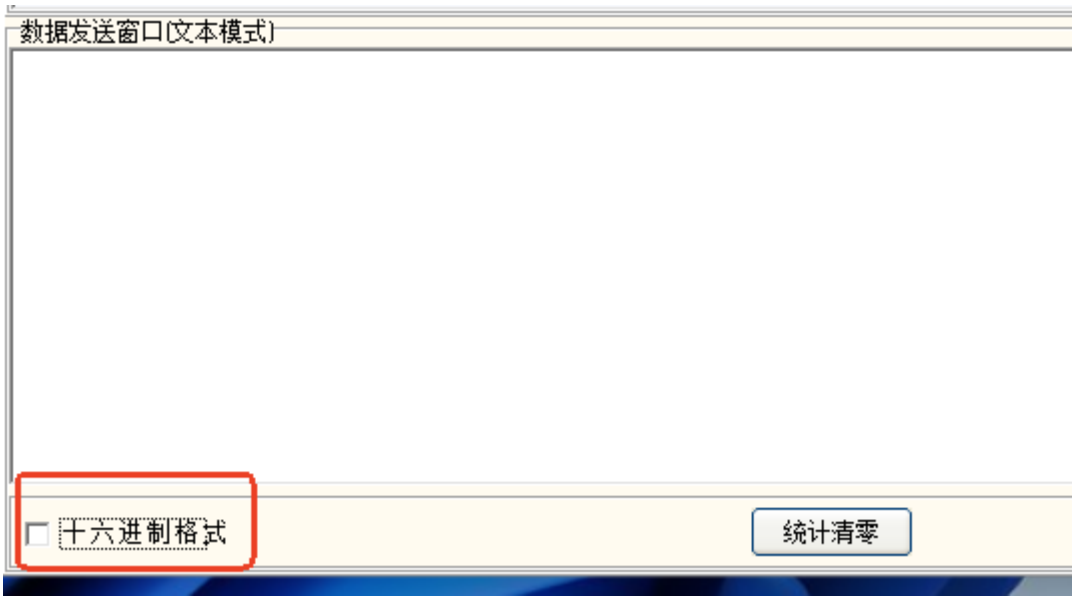


③ port keep same as code (8080 as example)



```
I (1703) UDP-SERVER: Got IP: 192.168.11.181
I (1703) UDP-SERVER: WiFi connected successfully!
I (1703) UDP-SERVER: UDP server created successfully!
I (1713) UDP-SERVER: Port 8080 bound successfully
I (1713) main_task: Returned from app_main()
```

④ don't check HEX format



⑤ enter message in text box

6.4 Normal Log — Server

```
I (2856) UDP-SERVER: Got IP: 192.168.11.181
I (2858) UDP-SERVER: WiFi connected successfully!
I (2860) UDP-SERVER: UDP server created successfully!
I (2862) UDP-SERVER: Port 8080 bound successfully
I (5000) UDP-SERVER: Receive computer messages: Hello ESP
I (5001) UDP-SERVER: Replied to the computer: I am ESP32 UDP Server!
```

```
I (380713) UDP-SERVER: Receive computer messages: Hello ESP
I (380713) UDP-SERVER: Replied to the computer: I am ESP32 UDP Server!
```

7. Common Problems

Symptom	Cause	Solution
Client `Send failed, errno=xxx`	Target IP unreachable or port not listening	Check PC and ESP32 on same LAN, confirm PC UDP listening on 8080
Server can't receive client data	ESP32 IP changed / firewall blocked	Check ESP32 actual IP via serial, PC firewall allows 8080 port
High receive latency	`vTaskDelay` too large	Ensure polling interval ≤ 20ms
`Failed to create UDP socket`	Socket descriptor exhausted	Check if forgot `close()` old Socket

Symptom	Cause	Solution
`Failed to bind port`	Port already in use	Change `UDP_SERVER_PORT`
Continuous sends but PC only receives partial	UDP no flow control, network congestion packet loss	Increase send interval / add ACK confirmation on sender (custom application layer protocol)
Chinese text garbled	Terminal encoding inconsistency	Check serial assistant and network debug tool encoding settings (UTF-8)